Final Project Report - Book Recommendation and Influence Analysis

By - Md Mushfiqur Rahman Khan

6240 - Large scale Parallel data processing

1. Introduction to the Problem

Which books and users have the most influence
In this project, i aim to analyze a large book data collection. I wish to understand which books and user of this platform has the most influence. I will be using a graph based technique such as page rank applied to a user-book network where ratings create connections between readers and books.

Which book will a particular user really like to read given his reading history.
Given information about users reading history, users location , similar users. We would like to know find out the next best book for the user by implementing a K-Nearest Neighbors algorithm that recommends books based on similarity to what the user already enjoys

Lastly We would like to generate interesting insights like age and location of this influential user or which authors are being read by this influential users

We implemented our solution using **Apache Spark**, a distributed computing framework, for several critical reasons:

Handling Scale: The dataset contains over 1.1 million ratings from 278,858 users across 271,379 books, making it too large for traditional single-machine processing. Spark's distributed architecture allows us to process this data efficiently across multiple nodes.

Graph Processing Capabilities: Spark's GraphFrames library provides essential graph algorithms like PageRank and community detection, enabling us to analyze the complex user-book network relationships that traditional SQL queries cannot capture effectively.

Integration with AWS EMR: Spark seamlessly integrates with Amazon EMR, allowing us to leverage cloud scalability for resource-intensive graph computations and KNN similarity searches across massive user bases

2. Data Description

Source: Books Dataset (Kaggle – https://www.kaggle.com/datasets/saurabhbagchi/books-dataset)
Size: ~124 MB across three files
The dataset contains:

1 **Books.csv:**
271,379 books with titles, authors, publication years, and publishers
Books Sample
ISBN;Book-Title;Book-Author;Year-Of-Publication;Publisher;Image-URL-S;Image-URL-M;Image-URL-L
0195153448;Classical Mythology;Mark P. O. Morford;2002;Oxford University Press;http://images.amazon.com/images/P/0195153448.01.THUMBZZZ.jpg;http://images.amazon.com/images/P/0195153448.01.MZZZZZZZ.jpg;http://images.amazon.com/images/P/0195153448.01.LZZZZZZZ.jpg
0002005018;Clara Callan;Richard Bruce Wright;2001;HarperFlamingo Canada;http://images.amazon.com/images/P/0002005018.01.THUMBZZZ.jpg;http://images.amazon.com/images/P/0002005018.01.MZZZZZZZ.jpg;http://images.amazon.com/images/P/0002005018.01.LZZZZZZZ.jpg

2 **Ratings.csv:**
1,149,780 explicit ratings (scale 1-10) linking users to books
Ratings sample:
UserID: 276744, BookID: 0155061224, Rating: 5
UserID: 276745, BookID: 0440178006, Rating: 7
Users.csv: 278,858 users with geographic locations and ages

3. **Users sample:**
UserID: 1, Location: "nyc, new york, usa", Age: 23
UserID: 2, Location: "stockton, california, usa", Age:

---

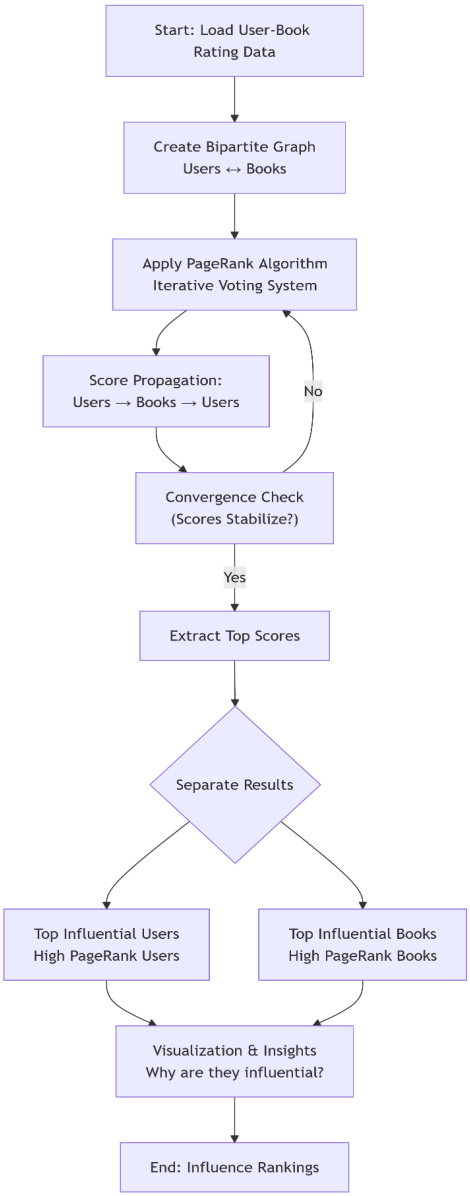
3. Sampling & Preprocessing

- Books with very few ratings has very few interaction with users, so including them can distort PageRank results hence i decided to filter books with less than 5 rating
- I decided to sample 30% of the data and used that to test page rank Algorithm and in future will use it to use the other algorithm
- This created a smaller subset, `sample_ratings`, which preserves the overall distribution of ratings while allowing faster testing and analysis, especially for the PageRank computation on the user-book graph.
- Created a Timer Class to compute time the Sparks take to run the program

---

4. Graph-Based Influence Analysis (PageRank)

- I computed **PageRank scores** to identify the most influential books and users in the dataset. I Used Page Rank Algorithm -
    - to find out the most influentia userl in this data set
    - to find out the most influential book in this data set
- I then prepared vertices and used user or book as column name.
- The edge was created from the rating data set.
- Used Graph frame and then used this for the page Rank Algorithm



- **Pseudocode**

```
# Prepare vertices
   userVertices = select distinct UserID as id, "user" as type
   bookVertices = select distinct ISBN as id, "book" as type
   vertices = union(userVertices, bookVertices)

   # Prepare edges
   edges = select UserID as src, ISBN as dst, BookRating as
   weight

   # Build graph and run PageRank
   graph = GraphFrame(vertices, edges)
   results = graph.pageRank(maxIter=5, resetProbability=0.15)

   # Extract top 10 books and users for display
   topBooks = results.vertices where type == "book" order by
   pagerank desc limit 10
   topUsers = results.vertices where type == "user" order by
   pagerank desc limit 10
```

- Output of most Popular User

```
UserID,pagerank
15957,0.6333521445954067
132171,0.6333521445954067
178199,0.6333521445954067
75149,0.6333521445954067
245534,0.6333521445954067
73091,0.6333521445954067
274549,0.6333521445954067
69042,0.6333521445954067
105536,0.6333521445954067
```

- Most Influential books

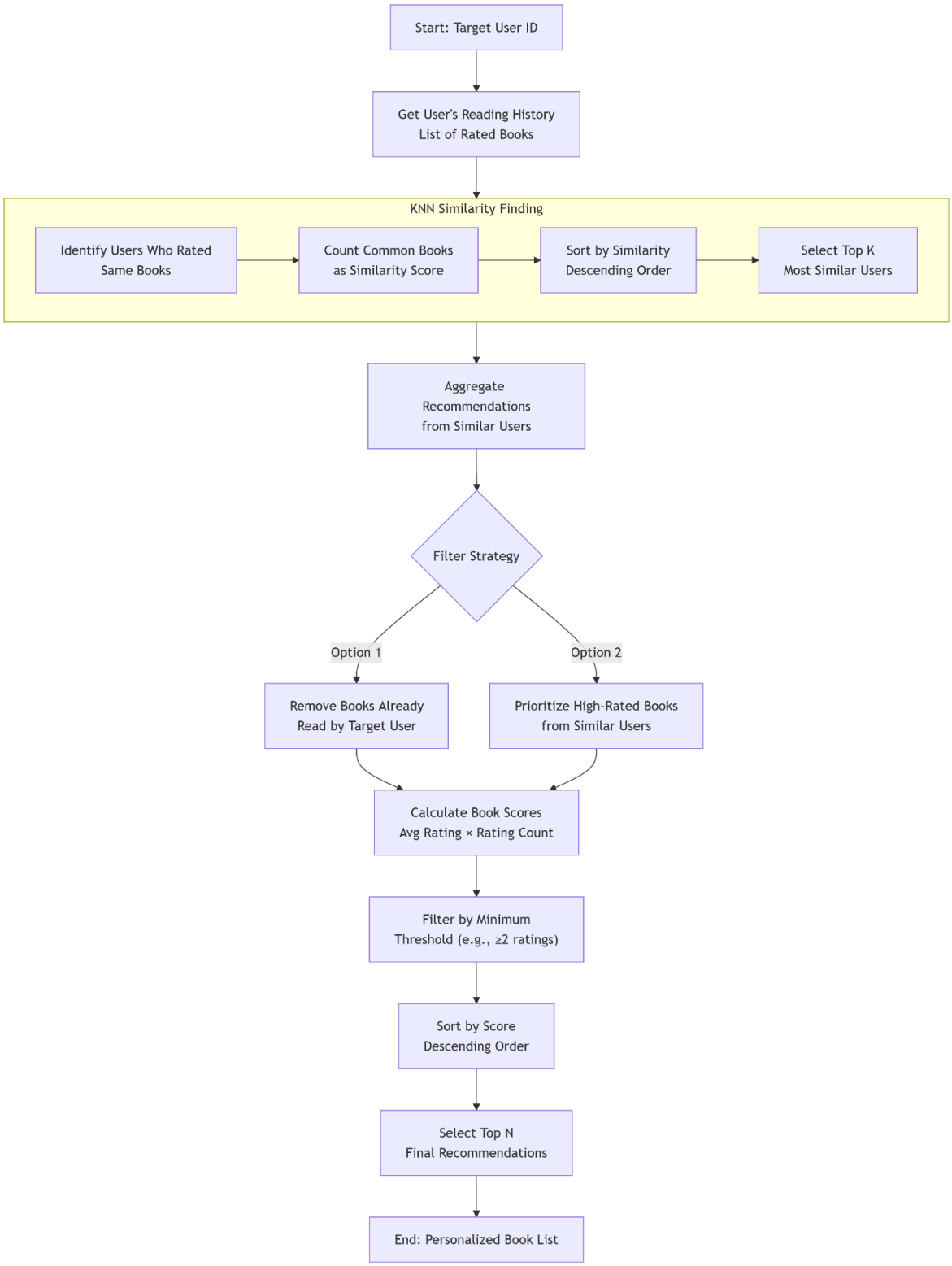
- output > top_books_pagerank > part-00000-87f9c96e-fea4-4850-9531-67d524f7803d-c000.csv

```csv
ISBN,BookTitle,BookAuthor,pagerank
0971880107,Wild Animus,Rich Shapero,106.64202263683063
0316666343,The Lovely Bones: A Novel,Alice Sebold,49.09971040159464
0385504209,The Da Vinci Code,Dan Brown,28.84507348188692
044023722X,A Painted House,John Grisham,23.149210407300547
0060928336,Divine Secrets of the Ya-Ya Sisterhood: A Novel,Rebecca Wells,21.96433662519392
0671027360,Angels &amp; Demons,Dan Brown,20.77125847500937
067976402X,Snow Falling on Cedars,David Guterson,19.87546662341369
0312195516,The Red Tent (Bestselling Backlist),Anita Diamant,18.517866118979658
0316601950,The Pilot's Wife : A Novel,Anita Shreve,18.29109962838118
0316769487,The Catcher in the Rye,J.D. Salinger,17.433746770931556
```

---

5. KNN Book recommendations

To find personalized recommendations for the most influential users i found in PageRank step of my I implemented a **K-Nearest Neighbor** Algorithm. This algorithm focuses of finding closest neighbour based on distance from the target user. This then uses the reading data of the neighbours to find the new books to recommend to target user. Then the algorithm proceeds to compile a unified recommendation output that contains recommendation for all the target users.


Start: Target User ID
Get User's Reading History
List of Rated Books
KNN Similarity Finding
Identify Users Who Rated
Same Books
Count Common Books
as Similarity Score
Sort by Similarity
Descending Order
Select Top K
Most Similar Users
Aggregate
Recommendations
from Similar Users
Filter Strategy
Option 1
Option 2
Remove Books Already
Read by Target User
Prioritize High-Rated Books
from Similar Users
Calculate Book Scores
Avg Rating × Rating Count
Filter by Minimum
Threshold (e.g., ≥2 ratings)
Sort by Score
Descending Order
Select Top N
Final Recommendations
End: Personalized Book List

- **Pseudocode**
  - **Step 1-**: Similar to PageRank, the ratings and books datasets are cleaned by normalizing the column names:

```
Dataset<Row> ratingsClean = ratingsDF
    .withColumnRenamed("UserID", "user_id")
    .withColumnRenamed("ISBN", "isbn")
    .withColumnRenamed("BookRating", "rating");

Dataset<Row> booksClean = booksDF
    .withColumnRenamed("ISBN", "isbn")
    .withColumnRenamed("Book-Title", "title")
    .withColumnRenamed("Book-Author", "author");
```

  - **Step 2**-: Identify Similar Users

```
similarUsers
    .groupBy("similar_user")
    .agg(count("*").alias("common_books"))
    .orderBy(desc("common_books"))
    .limit(kNeighbors);
```

  - **Step 3** : Aggregate Rating
    - Collect all books rated by the similar (neighbor) users

```
List<Row> similarUserList = similarUsers.collectAsList();
        List<String> similarUserIDs = new ArrayList<>();

        for (Row row : similarUserList) {
            similarUserIDs.add(String.valueOf(row.getInt(0)));
        }

        if (similarUserIDs.isEmpty()) {
            return spark.emptyDataFrame();
        }

        Dataset<Row> similarUserRatings = ratingsClean
```

```
.filter(col("user_id").isin(similarUserIDs.toArray()));
```

- Remove books that already has been read by the user
- Compute average,count of books ratings

```
Dataset<Row> aggregated = candidateBooks
    .groupBy("isbn")
    .agg(
        avg("rating").alias("avg_rating"),
        count("rating").alias("rating_count")
    );
```

- Filter books with atleast 2 rating and join it with the meta data
- Select the top books

```
Dataset<Row> recommendations = enriched
    .orderBy(desc("avg_rating"))
    .limit(topNRecommendations);
```

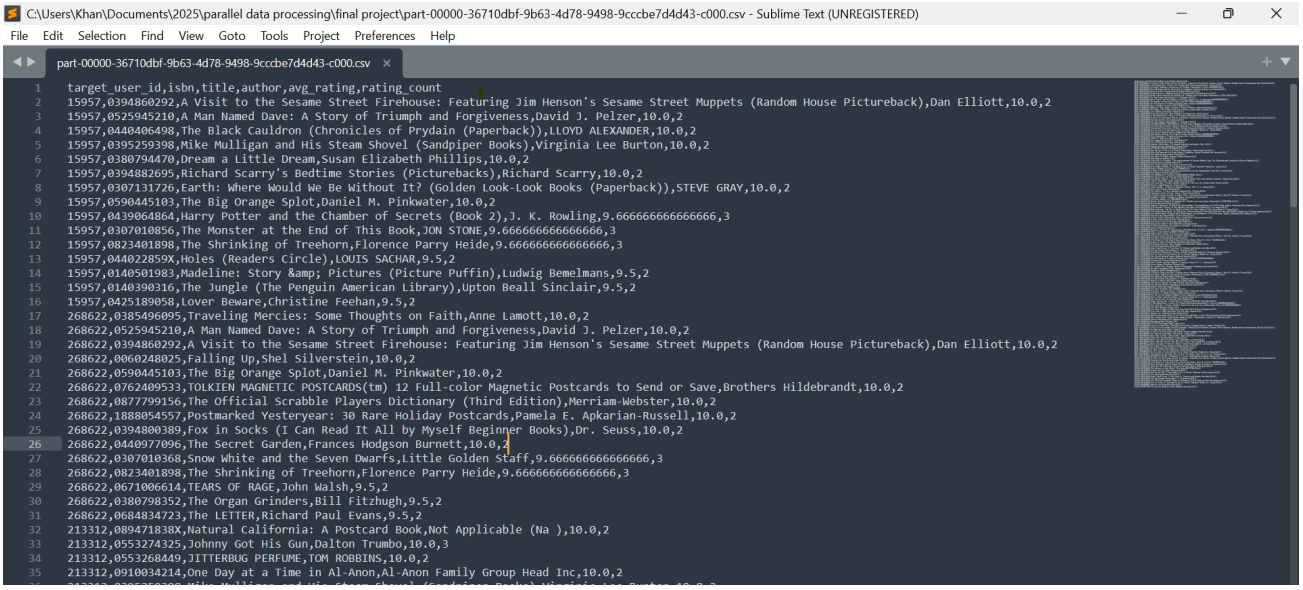
  - The algorithm then loops through all influential users.
  - The Algorithm then writes to the file

```
Dataset<Row> finalRecs = recommendations
    .withColumn("target_user_id", lit(targetUserID))
    .select("target_user_id", "isbn", "title", "author",
"avg_rating", "rating_count");
```

- **4.3 Output**

```
knn.saveCombinedRecommendations(
    combinedRecommendations,
    "s3://khan-final-project/output/all_knn_recommendations"
);
```

The following was saved to the file which included recommended book for all the most influential users.



---

7. Clever Solution.:

My Solution for PageRank Analysis used GraphFrames to transform the data into a graph structure. By treating books and users as nodes a bipartite graph was created. The rating here was used as edges. PageRank style traversal was done more efficiently thanks to scalable structure enabled by the transformation to bipartite graph. This also reduced shuffle overhead and improved parallelism of the algorithm.

The result from PageRank Analysis was used for K nearest neighbour input. This ensured the users have enough presence to generate good recommendations . To ensure Recommendation quality i set a minimum threshold of at least 2 ratings to be given to the book to be considered. Due to using Spark this was process was efficient even at large scale. I furthered improved the pipeline by caching intermediate DataFrames to reduce recomputation during iterative similarity calculations and used batch-processing logic to generate recommendations for multiple users at once.

---

7. Issues Encountered

CSV Seperator Issues - :

- Initial Spark errors occurred due to **column naming mismatches** (`Book-Rating` vs `BookRating`) and **CSV separator issues** (`;` instead of `,`).

- Initially I ran into Several Spark errors due to the naming Mismatch and CSV seperator Issue
- The file used ; instead of ,.
- Had to specify the Seperator by usint .option

```
Dataset<Row> booksDF = spark.read()
    .option("header", "true")
    .option("inferSchema", "true")
    .option("sep", ";")
```

AWS EMR Issues with Spark

I initially thought GraphFrames is part of Spark but after getting repeated step failures I checked the logs and realized in reality GraphFrames is an external library.

I had to use the following command in AWS UI to make it work

```
spark-submit \
--packages graphframes:graphframes:0.8.2-spark3.2-s_2.12 \
```

---

8. Key Observations

Results

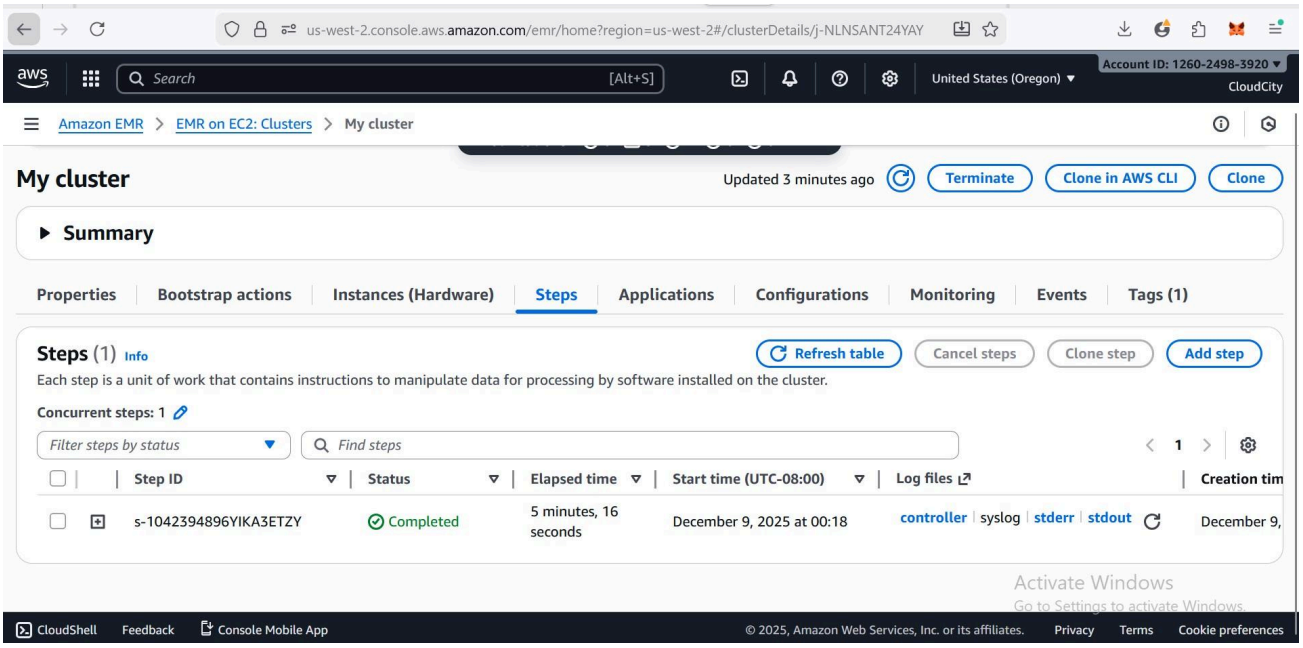
For Sample Data-:

- PageRank Analysis took 193425 ms (193.425 s) for the sampled data(30%) in local computer

- KNN recommendation took 3.276 seconds

```
output2 > timing.txt
PageRank Analysis took 193425 ms (193.425 s).
KNN Recommendations took 3276 ms (3.276 s).
```

For Complete DataSet -:




```
only showing top 20 rows


=== KNN Analysis Complete ===
Processed 10 influential users

=== Performance Summary ===
Data Filtering: 5.778 seconds
PageRank: 140.12 seconds
KNN Recommendations: 92.022 seconds
Total time: 237.92000000000002 seconds

=== Analysis Complete ===
```

Configuration of 1 main and 5 Core tasks

- PageRank Analysis took 140 second for the sampled data in local computer
- KNN recommendation took 92.022 seconds

Findings

This shows NO neighbour had rated more than 2 books

Title	Author	Avg Rating	Rating Count
A Visit to the Sesame Street Firehouse: Featuring...	Dan Elliott	10.0	2
A Man Named Dave: A Story of Triumph and Forgi...	David J. Pelzer	10.0	2
The Black Cauldron (Chronicles of Prydain...)	LLOYD ALEXANDER	10.0	2
Mike Mulligan and His Steam Shovel (Sandpiper...)	Virginia Lee Burton	10.0	2
Dream a Little Dream	Susan Elizabeth Phillips	10.0	2

The following Table shows the age and location of the most influential user, IT appears most of the users were from United States and had ages roughly 25-45

User-ID	Location	Age
15957	sydney, new south wales, australia	27.0
23571	burbank, california, usa	41.0
132171	milltown, wisconsin, usa	54.0
178199	amarillo, texas, usa	NaN
180981	middleburg, florida, usa	NaN
186039	tehran, tehran, iran	24.0
213312	baltimore, maryland, usa	47.0
234983	seattle, washington, usa	32.0

268622	upper township, new jersey, usa	36.0
271000	cambridge, massachusetts, usa	NaN

Observation

- Spark is a very useful and easy to use tool that does the heavy lifting of parallel processing for you
- The Time taken to run PageRank was substantially lower in EMR

---

9. Reflection

Working on this project so far has taught me How to use Spark to automate Mapping and reducing and how to Use spark to work with dataset and implement graph based technique like page rank that uses A bipartite graph.I had to face some Handling Column name issues and CSV separator problems but i was able to solve them. Using a sample dataset I was able to experiment and measure runtime and see if the page Rank would deliver Good Results. PageRank computation provided data that i can use to derive meaningful insights. Planning KNN-based recommendations highlighted the connection between graph analysis and collaborative filtering, strengthening my practical skills in large-scale data processing and applied analytics.

10. Individual Contributions

- All work for this project was completed individually by Md Mushfiqur Rahman Khan.

11. Reference

- **Apache Spark Documentation (2024)** - *MLlib: Collaborative Filtering*
  - https://spark.apache.org/docs/latest/ml-collaborative-filtering.html
  - ALS algorithm implementation
- **GraphFrames Documentation (2023)** - *Graph algorithms in Spark*
  - https://graphframes.github.io/graphframes/docs/_site/index.html
  - Label Propagation, PageRank implementations
- **AWS EMR Documentation (2024)** - *Managing dependencies on EMR clusters*

    - https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark-configure.html
    - Bootstrap actions for external libraries
- **Spark: The Definitive Guide (2020)** - *Bill Chambers & Matei Zaharia*
    - Chapter 30: Graph Analytics
    - Practical Spark deployment patterns